



Universidad Nacional Autónoma de México
Facultad de Ingeniería
División de Ingeniería Eléctrica



Carrera: Ingeniería en Computación

Profesor: Óscar Manuel Ruíz Hurtado

Curso: Sistemas Distribuidos

S05-MexiBanco

Alumno: Rodríguez Zamora Joshua

Fecha de entrega: 27/09/2026

Semestre: 2027-1

Grupo: 02

Paso 1. Copiar el repositorio:

Debido a que realicé la práctica y el reporte en dos días diferentes no cuento con la captura de la clonación del repositorio en mi máquina, sin embargo todas las demás actividades cuentan con lo obtenido a partir del mismo repositorio de mexiBanco.

Paso 2. Levántalo con Compose:

En este punto se levanta el servicio de app y db de mexibanco, y una vez hecho eso, se comprueba que ambos servicios estén correctamente levantados y saludables, listos para ser utilizados después.

```
jack1@LAPTOP-2CDLHG4C MINGW64 ~/mexi-banco ((v05.1))
$ docker compose up --build -d
#1 [internal] load local bake definitions
#1 reading from stdin 503B 0.0s done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 486B 0.0s done
#2 DONE 0.1s

#3 [internal] load metadata for docker.io/library/eclipse-temurin:21-jre-alpine
#3 ...

#4 [internal] load metadata for docker.io/library/eclipse-temurin:21-jdk-alpine
#4 DONE 1.4s

#3 [internal] load metadata for docker.io/library/eclipse-temurin:21-jre-alpine
#3 DONE 1.4s

#5 [internal] load .dockerignore
#5 transferring context: 2B done
#5 DONE 0.0s
```

imagen 1. Compose ejecutándose.

```
#20 unpacking to docker.io/library/mexi-banco-app:latest
#20 unpacking to docker.io/library/mexi-banco-app:latest done
#20 DONE 0.1s

#21 resolving provenance for metadata file
#21 DONE 0.0s
[+] up 3/3
✓ Image mexi-banco-app Built 2.7s
✓ Container mexi-banco-app-1 Running 0.0s
✓ Container mexi-banco-db-1 Healthy 0.6s
```

imagen 2. Servicios levantados.

```
jack1@LAPTOP-2CDLHG4C MINGW64 ~/mexi-banco ((v05.1))
$ docker compose ps

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS
mexi-banco-app-1	sha256:a0ef1af11a1b86ca7164f821add05610a282f71b5035d4ad0d2c6bff08e4bd51	"java -jar app.jar"	app	5 hours ago	Up 44 minutes (healthy)
mexi-banco-db-1	postgres:17-alpine	"docker-entrypoint.s..."	db	5 hours ago	Up 45 minutes (healthy)

imagen 3. Docker muestra Healthy en app y db.

Paso 3. Prueba los endpoints:

Colocaré un endpoint mínimo de cada sección para no mostrar todos.

1. Arriba y respondiendo:

En este conjunto de endpoints se busca mostrar que el servicio está arriba, funcionando y es utilizable, ya que podemos consultar y colocar datos correctamente.

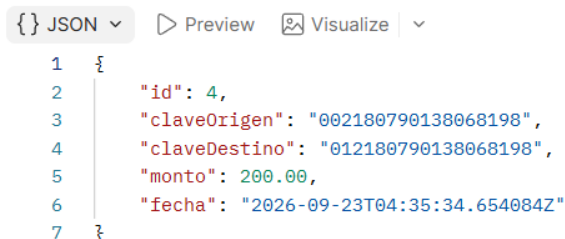


```
{ } JSON v
Preview Visualize v
1 {
2   "status": "UP",
3   "groups": [
4     "liveness",
5     "readiness"
6   ]
7 }
```

imagen 4. Endpoint 1.1. Revisión de los servicios activos.

2. Transferencia interna:

Para este endpoint se verifica el sistema de transferencias, abriendo la cuenta destino (de “Beto”) y haciendo que se transfieran \$200 pesos desde Ana, hasta Beto, después revisando los saldos de cada cuenta, dándonos cuenta que estos se modifican adecuadamente, restando \$200 pesos de Ana y sumando \$200 pesos a Beto.



```
{ } JSON v
Preview Visualize v
1 {
2   "id": 4,
3   "claveOrigen": "002180790138068198",
4   "claveDestino": "012180790138068198",
5   "monto": 200.00,
6   "fecha": "2026-09-23T04:35:34.654084Z"
7 }
```

imagen 5. endpoint 2.5. Transferencia interna realizada.

3. Idempotencia:

En esta parte revisamos la idempotencia, realizada a partir de la múltiple realización de una transferencia por SPEI de \$50 pesos, comprobando que al hacer la misma transferencia con la misma llave de idempotencia sólo se ve reflejada una sola transferencia entre el destinatario y el origen. Como se ve a continuación, pasando de un saldo de \$700.00 a \$750.00.

```

1  {
2      "id": 10,
3      "clabe": "002180790138269800",
4      "titular": "Ana",
5      "saldo": 750.00
6  }

```

imagen 6. Endpoint 3.3. Prueba de idempotencia.

4. El monolito por dentro:

En estos endpoints podemos visualizar los logs que guarda el monolito, desde el saldo inicial con el que se creó un usuario, hasta todos los movimientos que ha hecho este usuario y el id de la operación correspondiente.

```

1  [
2      {
3          "id": 24,
4          "clabeCuenta": "002180790138269800",
5          "tipo": "SPEI_ENVIADO",
6          "monto": -50.00,
7          "saldoResultante": 750.00,
8          "fecha": "2026-09-23T04:48:41.030659Z",
9          "referencia": "BBVA · 012180000000000099"
10     },
11     {
12         "id": 22,
13         "clabeCuenta": "002180790138269800",
14         "tipo": "TRANSFERENCIA_ENVIADA",
15         "monto": -200.00,
16         "saldoResultante": 800.00,
17         "fecha": "2026-09-23T04:38:03.802370Z",
18         "referencia": "a 012180790138269800"
19     },
20     {
21         "id": 20,
22         "clabeCuenta": "002180790138269800",
23         "tipo": "DEPOSITO",
24         "monto": 1000.00,
25         "saldoResultante": 1000.00,
26         "fecha": "2026-09-23T04:37:50.625082Z",
27         "referencia": "saldo inicial"
28     }
29 ]

```

imagen 7. Estado de Cuenta del origen.

Por otro lado, se tienen los avisos de los movimientos de dinero. Mostrándole al usuario un recibo de cada movimiento que ha hecho, usando días y horas.

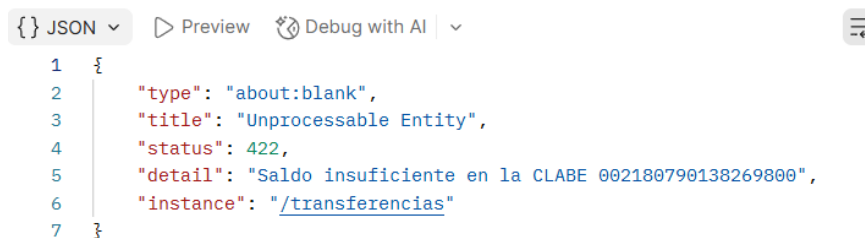


```
{
  "id": 13,
  "clabeCuenta": "002180790138269800",
  "mensaje": "SPEI_ENVIADO por 50, saldo resultante 750.00",
  "fecha": "2026-09-23T04:48:41.449024Z"
},
{
  "id": 11,
  "clabeCuenta": "002180790138269800",
  "mensaje": "TRANSFERENCIA_ENVIADA por 200, saldo resultante 800.00",
  "fecha": "2026-09-23T04:38:03.807517Z"
}
```

imagen 8. Control de movimientos.

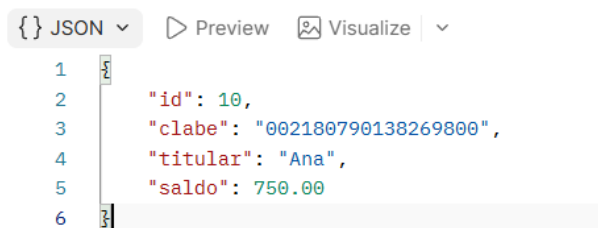
5. Cuando algo sale mal:

En este segmento se probaron varias posibilidades que dan error al crear elementos, ya sea porque se proporcionan datos no existentes, repetidos, que incumplen alguna regla (saldo) o falta algún dato. Finalmente se comprueba que al generar estos errores no se ejecuta ninguna acción y los valores quedan completamente iguales, sin cambios.



```
{
  "type": "about:blank",
  "title": "Unprocessable Entity",
  "status": 422,
  "detail": "Saldo insuficiente en la CLABE 002180790138269800",
  "instance": "/transferencias"
}
```

imagen 9. Ejemplo de error (saldo insuficiente).



```
{
  "id": 10,
  "clabe": "002180790138269800",
  "titular": "Ana",
  "saldo": 750.00
}
```

imagen 10. Datos del destino intactos ante errores.

Paso 4. Bájalo con y sin volumen:

- Docker compose down:

```
jack1@LAPTOP-2CDLHG4C MINGW64 ~/mexi-banco ((v05.1))
$ docker compose down
[+] down 3/3
✓ Container mexi-banco-app-1 Removed 0.4s
✓ Container mexi-banco-db-1 Removed 0.3s
✓ Network mexibanco Removed 0.3s
ddocdockdock
jack1@LAPTOP-2CDLHG4C MINGW64 ~/mexi-banco ((v05.1))
$ docker compose up -d
[+] up 3/3
✓ Network mexibanco Created 0.0s
✓ Container mexi-banco-db-1 Healthy 5.9s
✓ Container mexi-banco-app-1 Started 6.1s
```

imagen 11. Servicio abajo sin volumen.

Al usar Docker compose down y levantar el servicio inmediatamente con Docker compose up -d, podemos ver que las peticiones GET siguen devolviendo los mismos datos que teníamos antes de apagar el servicio. Esto debido a que sólo eliminamos el servicio y no los datos en sí.



```
{
  "id": 7,
  "claveOrigen": "002180790146117263",
  "claveDestino": "012180790146117263",
  "monto": 200,
  "fecha": "2026-09-23T06:48:48.870903323Z"
}
```

```
{
  "id": 30,
  "claveCuenta": "002180790146117263",
  "tipo": "TRANSFERENCIA_ENVIADA",
  "monto": -200.00,
  "saldoResultante": 800.00,
  "fecha": "2026-09-23T06:48:48.86785",
  "referencia": "a 012180790146117263"
}
```

- Docker compose down -v:

```
jack1@LAPTOP-2CDLHG4C MINGW64 ~/mexi-banco ((v05.1))
$ docker compose down -v
[+] down 4/4
✓ Container mexi-banco-app-1 Removed 0.4s
✓ Container mexi-banco-db-1 Removed 0.3s
✓ Network mexibanco Removed 0.3s
✓ Volume mexi-banco_mexibanco_datos Removed 0.1s
jack1@LAPTOP-2CDLHG4C MINGW64 ~/mexi-banco ((v05.1))
$ docker compose up -d
[+] up 4/4
✓ Network mexibanco Created 0.1s
✓ Volume mexi-banco_mexibanco_datos Created 0.0s
✓ Container mexi-banco-db-1 Healthy 5.9s
✓ Container mexi-banco-app-1 Started 6.1s
jack1@LAPTOP-2CDLHG4C MINGW64 ~/mexi-banco ((v05.1))
$
```

imagen 12. Servicios bajados con docker compose -v.

Al apagar el servicio con Docker compose down -v y levantarlo instantáneamente con Docker compose up -d, todos los endpoints se reinician. Esto es, que cada endpoint corre como si hubiera sido ejecutado por primera vez. Es muy notable cuando se ejecutan primero los endpoints de estado de cuenta y estos se muestran vacíos después de levantar de nuevo los servicios. Esto demuestra que los datos han sido eliminados de forma permanente y que la base de datos ha perdido todo. Por lo que las peticiones de GET devuelven errores o listas vacías, hasta que se reinicien todas las peticiones POST.

Sin datos en estados de cuenta:

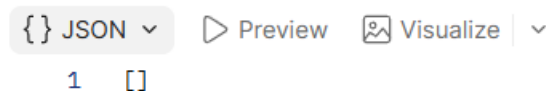


imagen 13. Estados de cuenta vacios.



imagen 14. No se encontró la cuenta de Ana.

Paso 5. Revisa el código:

Controller: En general estos archivos se encargan de manejar las peticiones web HTTP, traduciéndolas en instrucciones que después se ejecutan según sea el caso, ya sea revisando estados de cuenta, hacer transferencias internas o externas (con idempotencia como protección), generando movimientos y básicamente haciendo que cada servicio pueda generar una salida esperada.

Service: Tienen las reglas del banco, ya que cumplen con funciones que realizan modificaciones a la base de datos tomando en cuenta varios parámetros de forma segura y completa, haciendo uso de consideraciones como la llave de idempotencia en SPEI o la existencia de las cuentas bancarias, ya sea en transferencias internas o externas.

Repository: Es el tipo de archivo que se encarga de hablar con la base de datos, usando a las demás clases como ayuda para modificar a la db. Pueden guardar, buscar y eliminar.

Las entidades: Estos son los códigos base de donde se generan tablas en la base de datos de cada función, ya sea SPEI, transferencias, movimientos, notificaciones y cuenta. Además de que se crean los constructores que al ser llamados devuelven como parámetros los históricos de cada operación, teniendo campos, **por ejemplo**, como el id de la operación, la CLABE de origen, la CLABE de destino, el monto y la fecha. En general son los códigos que se encargan de establecer tablas y funciones que después los demás códigos utilizarán.

Paso 6. Estudia el compose y el Dockerfile:

- **Dockerfile:** Cuenta con 2 etapas, una de construcción y otra de ejecución.

Construcción: En la etapa de construcción podemos encontrar toda la configuración de herramientas que necesitamos para ejecutar después la imagen. Empezando por descargar la imagen del sistema operativo (Alpine), que debido a que es muy ligero, nos permite ahorrar memoria y obtener la Java Developer Kit. Entorno que nos permitirá correr nuestros códigos en java, y para facilitar la llamada a este SO, colocaremos de apodo builder al mismo.

Después, colocamos el comando **WORKDIR /app**, con lo que creamos y entramos al directorio /app en el contenedor, todo lo que hagamos, funciona aquí

Acto seguido, tenemos comandos de **COPY**, que prácticamente descargan la configuración de Maven, una herramienta de automatización de compilación y gestión de proyectos que aplicaremos a Java. Aunado a esto, descargaremos las dependencias de internet con **RUN ./mvnw dependency:go-offline -B** para que Docker usé la caché y no vuelva a descargar todo, ahorrando mucho tiempo.

Ahora, con el comando **COPY src ./src** copiamos el código fuente de la aplicación a src para finalmente compilar el código y a la vez empaquetarlo en un archivo .jar con el comando **RUN ./mvnw clean package -DskipTests.**

Ejecución: En la segunda etapa, la etapa de ejecución, tenemos el uso nuevamente de alpine con el comando **FROM eclipse-temurin:21-jre-alpine**, sólo que esta vez usando el JRE (Java Runtime Environment), utilizando sólo las funciones necesarias para correr Java, evitando usar un compilador y de esta forma ahorrando memoria.

Posteriormente, volvemos a entrar a **WORKDIR /app** para configurar la imagen de ejecución con el comando **COPY --from=builder /app/target/*.jar app.jar**, que nos sirve para utilizar el empaquetado .jar previamente compilado en la etapa anterior (el builder) y lo usamos en este nuevo entorno como app.jar.

Ahora, con la aplicación en el entorno, abriremos el puerto 8080 para que el contenedor pueda escuchar peticiones por ahí.

Finalmente, con el comando **ENTRYPOINT ["java", "-jar", "app.jar"]**, hacemos que al encender el contenedor, se ejecute la aplicación Java.

- **Docker Compose:**

Para DB:

En Docker compose definimos dos servicios que son: db y app. Además de definir la comunicación de las redes y la recopilación de información.

1. **Imagen:** Primero se define la imagen con postgres:17-alpine y se crea un entorno para la base de datos de postgres, poniendo de nombre mexibanco, al igual que un usuario y contraseña con el mismo nombre "mexibanco".
2. **Persistencia:** Se crea un volumen con una ruta en donde queremos guardar los datos, permitiendo la persistencia de datos, lo que permite mantenerlos a pesar de borrar el contenedor.
3. **Prueba de vida:** Para comprobar que el contenedor se crea y funciona adecuadamente se utiliza una prueba de vida con healthcheck, que hace un test, cada 5 segundos, y coloca limites en la autenticación. Permitiendo 5 intentos de autenticación y no más de 3 segundos de timeout.

Para app:

1. **Build:** Se busca el dockerfile anterior para construir la app, y con ello conectamos el puerto 8080 de la computadora con el puerto 8080 del contenedor mediante ports: 8080:8080, de esta forma comprobaremos el localhost:8080 desde el navegador.
2. **Environment:** Se añade configuración a la base de datos, colocando un nombre de host, puerto (5432), nombre, usuarios, contraseña y la condición de que se cumpla la prueba de vida de la base de datos previamente establecida. Si postgres no responde al healthcheck, la app no correrá, esto para evitar errores.
3. **Healthcheck:** Genera una prueba de vida para la app.

Para la red y el volumen:

1. **Networks: default: name: mexibanco.** Modifica el nombre de la red por defecto para que todos los contenedores dentro estén conectados bajo esta red reconocible.
2. **Volumes: mexibanco_datos:** Hace que Docker cree esté volumen y se pueda administrar.

Conclusión:

En esta sesión juntamos varios conocimientos básicos sobre Docker, volúmenes y todo el conocimiento acumulado para añadirle una aplicación funcional en la cual pudimos interactuar para además comprobar como todo el contenido visto hasta ahora es aplicable para levantar una aplicación del tipo cliente-servidor. En general puedo decir que esta práctica es la puerta de entrada para familiarizarnos con las aplicaciones distribuidas y microservicios.